BOOLEAN ALGEBRA & K-MAPS

SCSC1013 - COMPUTER SYSTEM & ORGANIZATION

Module 2 · Canonical Logic & Minimisation · Weeks 4–6

Dr Zuriahati Mohd Yunos · BSE (Hons.)

Gates become easier when we give them a language

01

GATE NETWORK

Physical relationships

02

BOOLEAN EXPRESSION

Compact symbolic rule

03

OUTPUT DECISION

Predict for any input

Language lets us reason before we build.

WEEK 4 OUTCOMES

This week: describe, simplify, transform and verify

01 **Describe**
Use Boolean variables and operators to represent a circuit.

02 **Simplify**
Apply named laws and justify every transformation.

03 **Transform**
Use De Morgan correctly across complemented groups.

04 **Verify**
Make circuit, expression and truth table agree.

WEEK 4 · MEETING 1

Block 1 · Boolean language

How can one expression describe an entire gate network?

Variable

- A symbol that represents a logical quantity
- Usually italic uppercase (*A*, *B*, *C*, *D*)
- A single variable can have a 1 or 0 value

Complement

- The inverse of a variable
- indicated by an overbar $\bar{A}$ or prime A'
- If $A = 1$, then $\bar{A} = 0$

Literal

- = both variable and its complement in expression
- $\bar{A} + B + C$ → 3 literals

Example:

$$f = x' + y' + x'y \qquad \textbf{(Eq 1)}$$

$$f = \bar{x}\bar{y} + \bar{x}y \qquad \textbf{(Eq 2)}$$

Total Number of Literals = 4 which are *x'*, *y'*, *x'* and *y*

Total Number of variables = 2 which are *x* and *y*

Total Number of terms = 2 which are *x'y'* and *x'y*

Boolean variables name two-state signals

VARIABLES

A, B, C ∈ {0,1}

Variables name signals.

0 and 1 are logic states, not counts.

STATE

0

LOW
FALSE
inactive

STATE

1

HIGH
TRUE
active

Always state the application meaning: D = 1 means ‘door open’.

NOT first, then AND, then OR

01

NOT

A'

02

AND

AB or A·B

03

OR

A+B

F = A'B + AC

two product terms
combined by OR

+ means OR — not arithmetic addition.

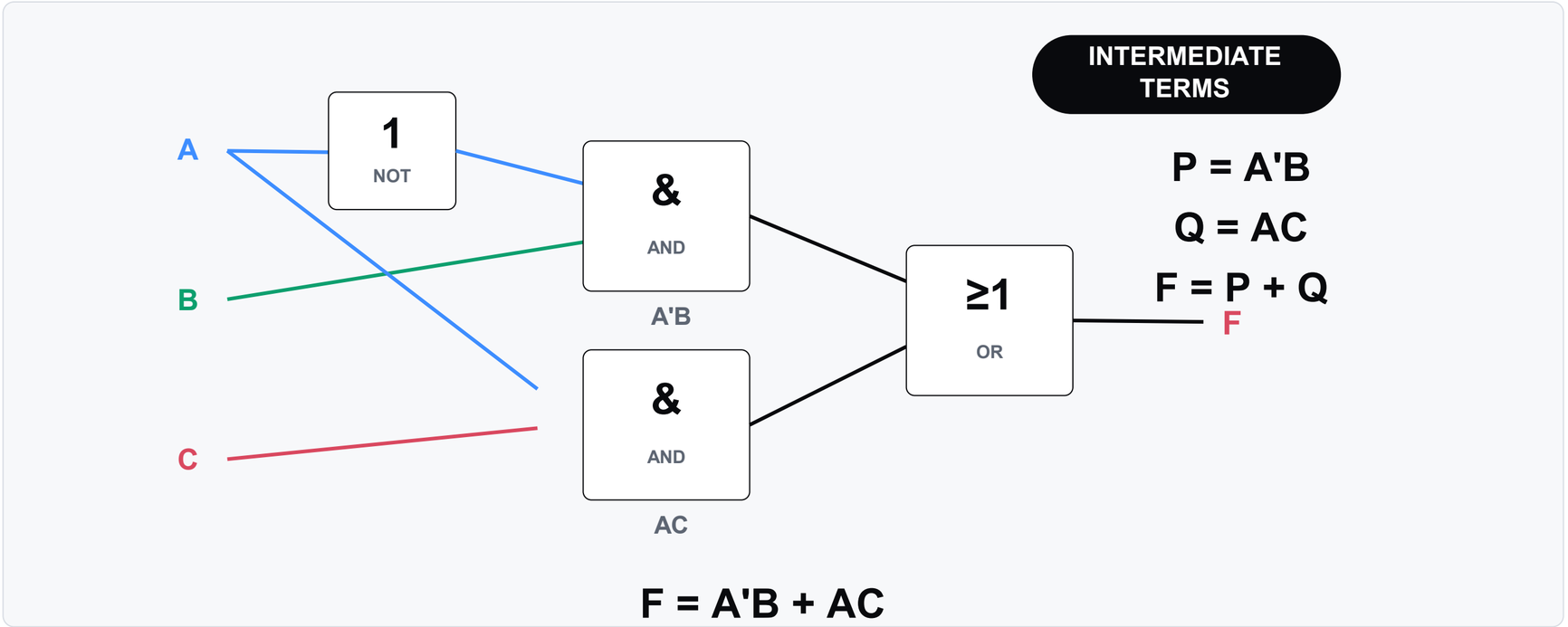
Evaluate one operation at a time

Given A=0, B=1, C=0

01	02	03	04
A'	A'B	AC	F
1	1·1 = 1	0·0 = 0	1+0 = 1

F = A'B + AC = 1

The expression reveals the gate structure



PAIR ACTIVITY · 13 MIN

Your turn: evaluate before you simplify

1. For A=1, B=0, C=1: find A', A'B, AC and F.
2. For A=1, B=1, C=0: find A', A'B, AC and F.
3. For each case, identify which product path—if any—drives F HIGH.

Record intermediate values. A final answer alone is not enough.

BLOCK 1 CHECK

Checkpoint: trace the final operator

A=0, B=0, C=1 **F = A'B + AC**

01 **Calculate F**
Show A', A'B and AC.

02 **Name the final operation**
Which operator combines the two terms?

Guardrail: Boolean + means OR.

WEEK 4 · MEETING 1

Block 2 · Core Boolean laws

Which expressions collapse without changing their truth tables?

Four law families remove redundant logic

LAW	OR FORM	AND FORM	IDEA
Identity	A + 0 = A	A.1 = A	neutral input
Null element	A + 1 = 1	A.0 = 0	forcing input
Idempotency	A + A = A	A. A =A	duplicate input
Complement	A + A’ = 1	A.A’ = 0	opposite states

Also remember involution: (A')' = A

Distributive and covering laws reshape logic

RESHAPE

DISTRIBUTIVE

$$A(B+C) = AB+AC$$

$$A+BC = (A+B)(A+C)$$

REMOVE

COVERING (ABSORPTION)

$$A+AB = A$$

$$A(A+B) = A$$

Name the law at every step.

The covering theorem removes a redundant condition

EXAMPLE 1

$$A + AB = A(1+B) = A$$

Whenever A=1, the expression is already 1. The AB term cannot add a new 1-row.

Also called absorption: A(A+B) = A

A defensible simplification names every step

AB + AB' + A'B	START
A(B+B') + A'B	DISTRIBUTIVE
A + A'B	COMPLEMENT + IDENTITY
(A+A')(A+B)	DISTRIBUTIVE
A+B	COMPLEMENT + IDENTITY

Rules of Boolean Algebra

1	$A + 0 = A$
2	$A + 1 = 1$
3	$A \bullet 0 = 0$
4	$A \bullet 1 = A$
5	$A + A = A$
6	$A + \overline{A} = 1$
7	$A \bullet A = A$
8	$A \bullet \overline{A} = 0$
9	$\overline{\overline{A}} = A$
10	$A + AB = A$
11	$A + \overline{A}B = A + B$
12	$(A + B)(A + C) = A + BC$

GUIDED PRACTICE · 12 MIN

Simplify and name each law

1. X + X'Y
2. X(X' + Y)
3. (X + Y)(X + Y')

Write the law beside every transformation.

Boolean algebra does not use ordinary cancellation

USE

BOOLEAN

X+X = X

XX' = 0

+ means OR

AVOID

DO NOT CANCEL

AB = AC

⇏ B = C

A may be 0

Use Boolean laws or a truth table—not subtraction or division.

BLOCK 2 CHECK

Checkpoint: defend P + P'Q = P + Q

$$P + P'Q = P + Q$$

01 Transform
Show a valid law at every line.

02 Verify
Explain why the final function is equivalent.

No ordinary-algebra cancellation.

Rules of Boolean Algebra

1	$A + 0 = A$
2	$A + 1 = 1$
3	$A \cdot 0 = 0$
4	$A \cdot 1 = A$
5	$A + A = A$
6	$A + \overline{A} = 1$
7	$A \cdot A = A$
8	$A \cdot \overline{A} = 0$
9	$\overline{\overline{A}} = A$
10	$A + AB = A$
11	$A + \overline{A}B = A + B$
12	$(A + B)(A + C) = A + BC$

WEEK 4 · MEETING 2

Block 3 · De Morgan's laws

Complementing a group changes both its literals and its operator.

De Morgan gives two transformation rules

AND → OR

COMPLEMENT A PRODUCT

$$(AB)' = A' + B'$$

OR → AND

COMPLEMENT A SUM

$$(A+B)' = A'B'$$

Move the complement to every literal · swap AND ↔ OR

Bubble movement explains NAND and NOR equivalence

EQUIVALENT

NAND

$(AB)' = A' + B'$

AND with output bubble
↔ OR with input bubbles

EQUIVALENT

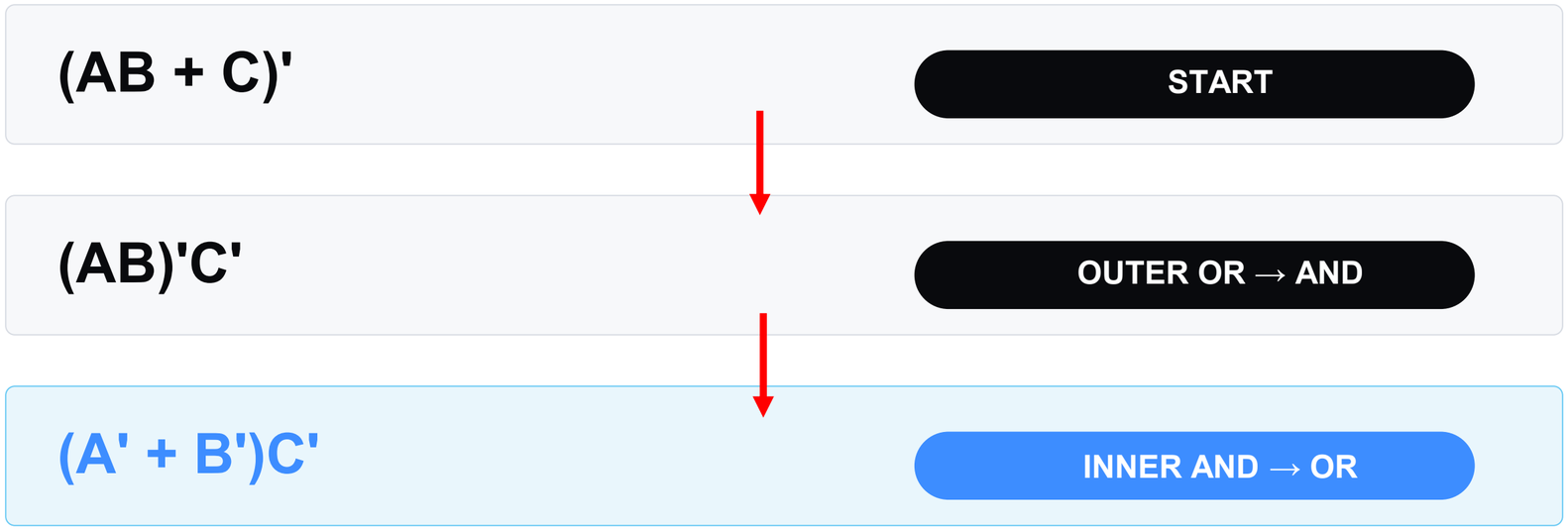
NOR

$(A+B)' = A'B'$

OR with output bubble
↔ AND with input bubbles

Push a bubble through a gate: invert every input and swap AND ↔ OR.

A group complement applies to the whole group



Both actions matter: complement every literal and swap the operator.

PAIR PRACTICE · 13 MIN

Transform without skipping the group boundary

1. (X + Y + Z)'
2. (XY + Z)'
3. (X + Y')'

Show where the group bar begins and ends.

Check every complement and operator change

A	B	(A+B)’	A’B’	(AB)’	A’+B’
0	0	1	1	1	1
0	1	0	0	1	1
1	0	0	0	1	1
1	1	0	0	0	0

Equivalent columns match on every row.

BLOCK 3 CHECK

Checkpoint: which expression equals (PQ)'?

A

P'Q'

B

P' + Q'

C

P + Q

Test one discriminating input before you commit.

Equivalent logic can be viewed five ways



A shorter expression is useful—but equivalence must still be proved.

Name intermediate signals before the output

01

X = A+B

first OR path

02

Y = A+C

second OR path

03

F = XY

final AND

F = (A+B)(A+C)

Equivalent representations must agree on every input row

A	B	C	X=A+B	Y=0+C	F=XY
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	1	0	0
0	1	1	1	1	1
1	0	0	1	1	1
1	0	1	1	1	1
1	1	0	1	1	1
1	1	1	1	1	1

Trace intermediates first; diagnose the earliest mismatch.

The same function can need fewer operations

3 OPERATIONS

ORIGINAL

F=(A+B)(A+C)

2 OR paths
1 AND output

2 OPERATIONS

SIMPLIFIED

F=A+BC

1 AND path
1 OR output

Equivalent output sequence: 0, 0, 0, 1, 1, 1, 1, 1

END-TO-END PRACTICE · 12 MIN

Complete the representation cycle

1. Derive F=UV from U=A+B and V=A+B'.
2. Build the complete A,B,U,V,F truth table.
3. Simplify, redraw and verify every row.

Complete every representation before comparing with another pair.

Peer verification requires all rows to agree

01	**Same input order** Do both tables enumerate inputs identically?
02	**All rows present** No combination may be skipped.
03	**Intermediate nodes traceable** Can each output be recomputed?
04	**Final outputs identical** Original and simplified columns match.

Shorter ≠ proven. Matching behaviour = proven.

EXIT TICKET · 3 MIN

Week 4 checkpoint: prove, do not merely shorten

FUNCTION

$$H = A + AB + A'B$$

Simplify.
Name each law.
Verify all 4 rows.

A	B	ORIGINAL	SIMPLIFIED
0	0	?	?
0	1	?	?
1	0	?	?
1	1	?	?

Next: every 1-row becomes a minterm.

WEEK 5 · MEETING 1

Week 5 · Canonical logic representation

A complete truth table is stronger evidence than a few successful tests.

A few successful tests are not complete evidence

PARTIAL

ONE TEST PASSES

Evidence for one input only.

The remaining rows are unknown.

COMPLETE

ALL ROWS MATCH

Every input combination agrees.

The implementation is verified.

Verification asks whether two functions agree everywhere.

WEEK 5 OUTCOMES

This week: analyse, canonicalise, implement and test

01 **Analyse**

Derive expressions and complete truth tables from a combinational circuit.

02 **Canonicalise**

Convert every output-1 row into a minterm and canonical SOP.

03 **Implement**

Map product terms to an AND–OR network.

04 **Test**

Verify the circuit against every source row.

Anchor function: A selects which input reaches F

$$F = A'B + AC$$

SELECT B

A = 0

A'=1 → F follows B

SELECT C

A = 1

A=1 → F follows C

A combinational output uses present inputs only

SCOPE

$$F(t) = f[A(t), B(t), C(t)]$$

Change an input and the output rule is re-evaluated. There is no stored past value in this week.

No feedback · no state · no memory

Use the same analysis path every time

01

INPUTS / OUTPUT

Name and define

02

INTERMEDIATE SIGNALS

Label each node

03

EXPRESSION

Substitute systematically

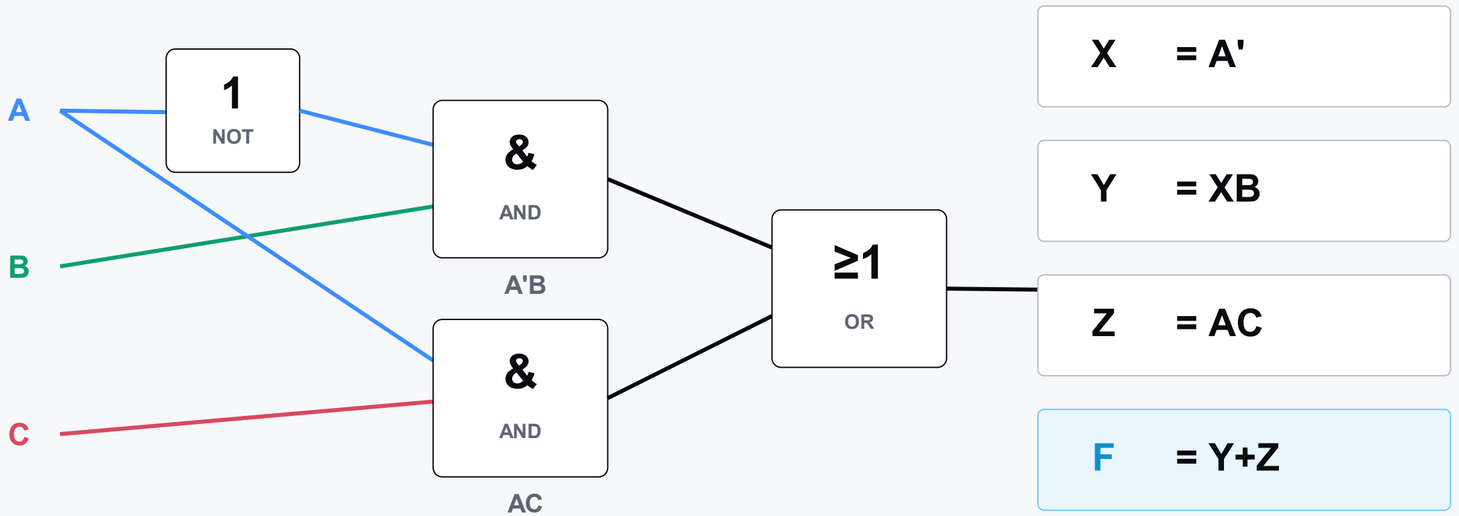
04

TRUTH TABLE

Enumerate all rows

Traceability is the goal: every table column has an origin.

Intermediate nodes make the circuit traceable



The labelled network becomes one expression

$F = Y + Z$ — OUTPUT NODE

↓

$F = XB + AC$ — SUBSTITUTE Y AND Z

↓

$F = A'B + AC$ — SUBSTITUTE X

Do not skip a named node when the network is multilevel.

Enumerate inputs systematically from 000 to 111

ROW	0	1	2	3	4	5	6	7
A	0	0	0	0	1	1	1	1
B	0	0	1	1	0	0	1	1
C	0	1	0	1	0	1	0	1

A is MSB · C is LSB · C changes fastest

INDIVIDUAL → PAIR · 10 MIN

Complete the intermediate columns

A	B	C	X=A'	Y=XB	Z=AC	F
0	0	0	?	?	?	?
0	0	1	?	?	?	?
0	1	0	?	?	?	?
0	1	1	?	?	?	?
1	0	0	?	?	?	?
1	0	1	?	?	?	?
1	1	0	?	?	?	?
1	1	1	?	?	?	?

Find the first mismatching intermediate column before checking F.

The selector is 1 on four input rows

A	B	C	A'	A'B	AC	F
0	0	0	1	0	0	0
0	0	1	1	0	0	0
0	1	0	1	1	0	1
0	1	1	1	1	0	1
1	0	0	0	0	0	0
1	0	1	0	0	1	1
1	1	0	0	0	0	0
1	1	1	0	0	1	1

F=1 for 010, 011, 101 and 111

BLOCK 1 CHECK

Checkpoint: trace row 110

A B C = 1 1 0 **X=A' · Y=XB · Z=AC · F=Y+Z**

01	02	03	04
X	Y	Z	F
?	?	?	?

WEEK 5 · MEETING 1

Block 2 · Minterms and canonical SOP

One output-1 row becomes one complete product term.

A minterm contains every input variable

UNCOMPLEMENTED

BIT = 1

use the literal
A, B or C

COMPLEMENTED

BIT = 0

use the complement
A', B' or C'

A minterm is an implicant with one literal for every input.

The minterm index is the row's binary value

01	02	03	04
ABC	**BINARY VALUE**	**LITERALS**	**MINTERM**
101	$101_2 = 5$	AB'C	m5

101 → AB'C = m5

Three variables define eight minterms

INDEX	ABC	MINTERM
m0	000	A'B'C'
m1	001	A'B'C
m2	010	A'BC'
m3	011	A'BC
m4	100	AB'C'
m5	101	AB'C
m6	110	ABC'
m7	111	ABC'

Variable order: A is MSB, C is LSB.

The selector's 1-rows identify its minterms

010	→	m2	→	A'BC'
011	→	m3	→	A'BC
101	→	m5	→	AB'C
111	→	m7	→	ABC

$$F = \Sigma m(2,3,5,7)$$

Canonical SOP lists every 1-row explicitly

CANONICAL SOP

$$F = \Sigma m(2,3,5,7)$$

F = A'BC' + A'BC + AB'C + ABC

Every product term names one complete 1-row.

The same function has canonical SOP and POS

1-ROWS

CANONICAL SOP

$Y = A'B + AB = \Sigma m(1,3)$

use minterms where Y = 1

0-ROWS

CANONICAL POS

$Y = (A+B)(A'+B) = \Pi M(0,2)$

use maxterms where Y = 0

Both simplify to Y = B · canonical means standard form.

BLOCK 2 CHECK · 12 MIN

Convert 1-rows into a canonical SOP

1. Write the minterm index for 001, 100 and 110.
2. Write the full product term for each row.
3. Combine them as Σm(…) and expanded canonical SOP.

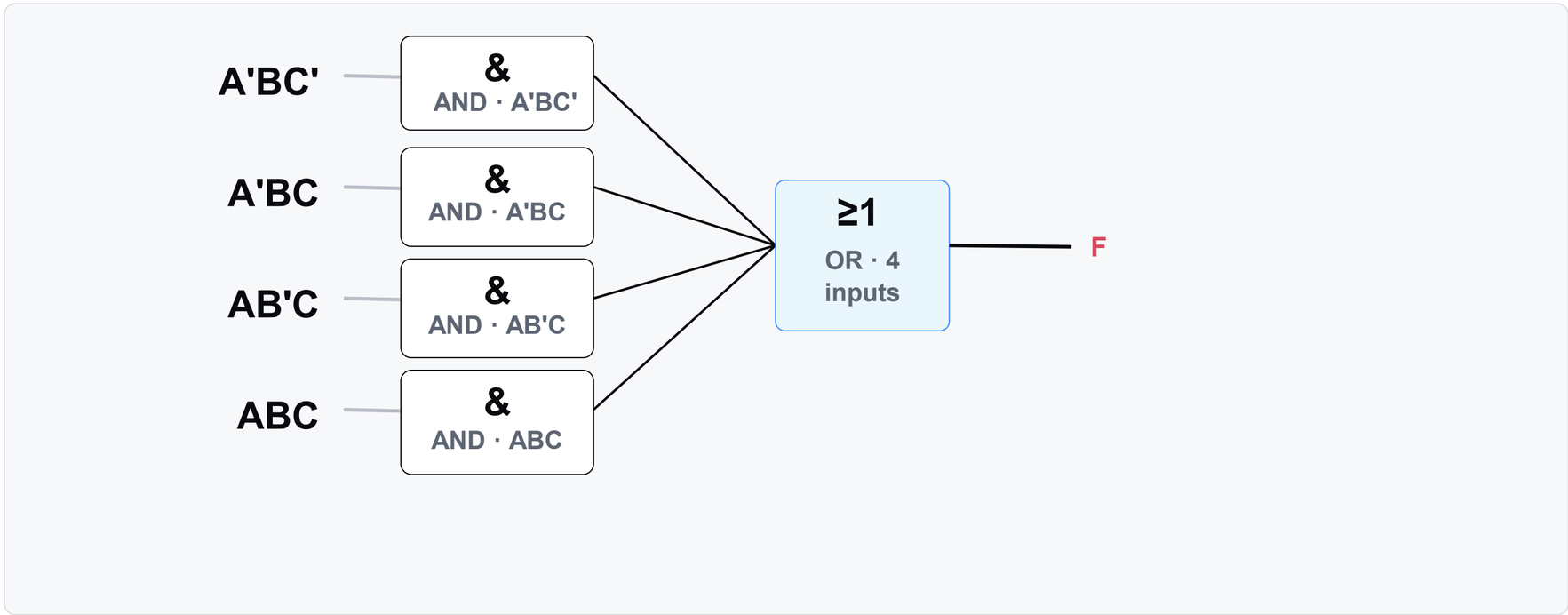
Audit: every product term contains A, B and C.

WEEK 5 · MEETING 2

Block 3 · Canonical SOP implementation

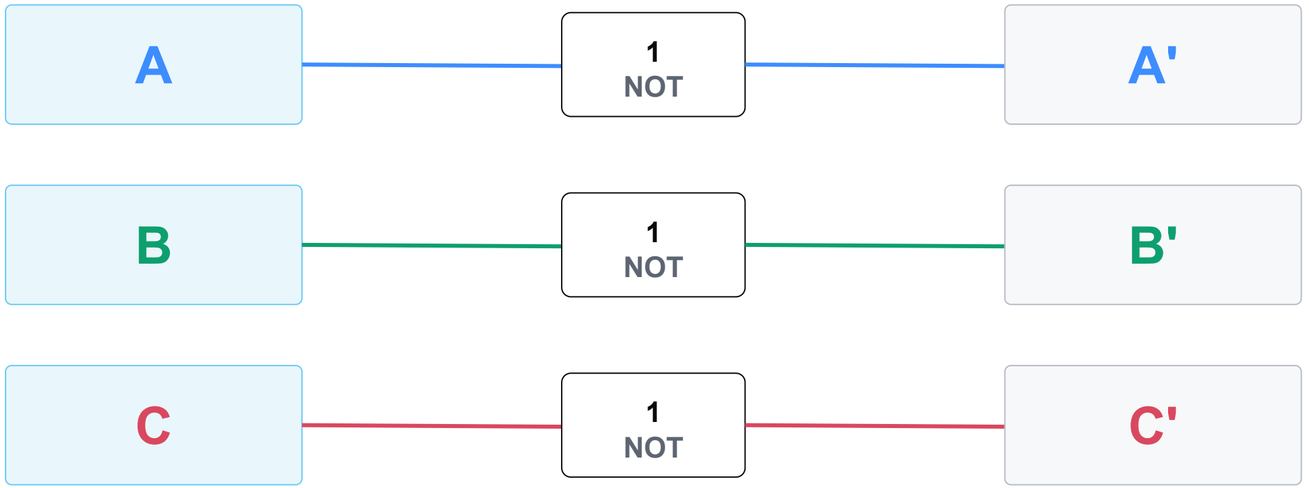
Product terms become AND gates; their sum becomes OR.

The canonical circuit has four product paths



Each AND path detects one minterm; OR collects all valid 1-rows.

Prepare complemented inputs once and reuse them



These six rails feed every canonical product term.

Canonical cost is easy to count

4	12	4
PRODUCT TERMS	LITERAL OCCURRENCES	OR INPUTS

Counting now makes Week 6 reduction measurable.

Verification compares the circuit with its source table

01

APPLY INPUT

000 → 111

02

PREDICT

from source table

03

OBSERVE

circuit output

04

COMPARE

must match

8 matching rows = verified · 1 mismatch = diagnose

SIMULATION LAB · 12 MIN

Build and test the canonical selector

1. Build AND paths for m2, m3, m5 and m7.
2. Predict F for 010, 100, 101 and 111.
3. Run all eight cases and log any mismatch.

Prediction first. Simulation second. Equivalence needs all eight rows.

BLOCK 3 CHECK

Checkpoint: passing one test is not equivalence

CLAIM

“The circuit works for 101.”

What can you conclude? Only that one row passed. Equivalence requires every row from 000 through 111.

SOP is mastery · POS is recognition only

WEEK 5 · MEETING 2

Block 4 · Specification to function

Begin with a requirement—not an existing expression.

Resolve variables and active levels first

A = **system armed**

1 means active

D = **door open**

1 means active

W = **window open**

1 means active

Q = **alarm active**

1 means active

Requirement: Q=1 when A=1 and (D=1 or W=1).

The specification determines the complete table

A	D	W	Q
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Active rows: 101, 110, 111

The alarm's canonical SOP names all active rows

ALARM FUNCTION

$$Q = \Sigma m(5,6,7)$$

$$Q = AD'W + ADW' + ADW$$

The canonical form mirrors the table exactly.

Implementation and tests must match the requirement

01	**Unarmed + entry open** Q must remain 0.
02	**Armed + both closed** Q must remain 0.
03	**Armed + one open** Q must become 1.
04	**Armed + both open** Q must become 1.

Then verify all eight rows—not only these representative cases.

INDIVIDUAL DESIGN · 10 MIN

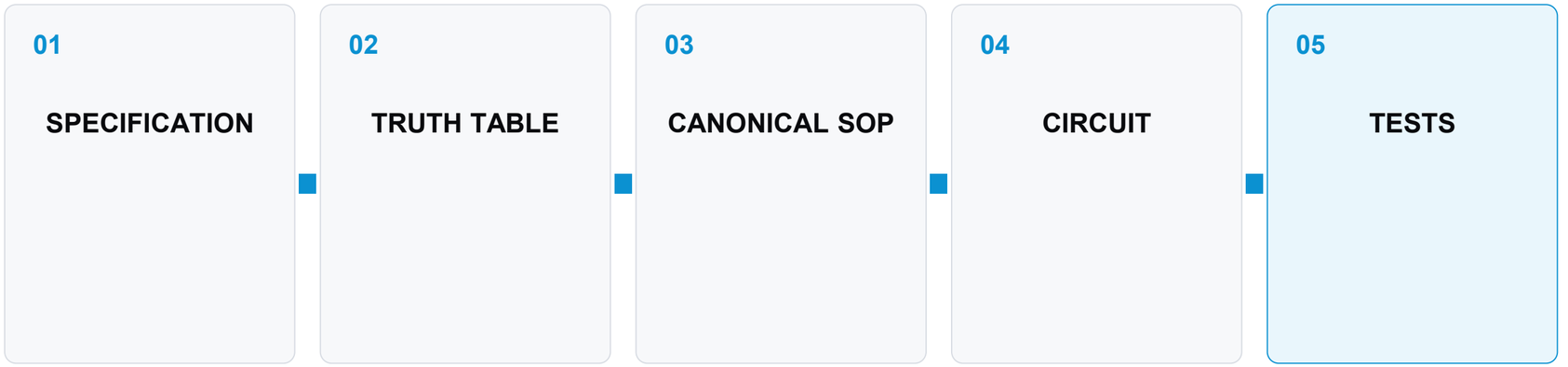
Mini-design: exactly one key authorises access

1. Define A, K, R and G with active levels.
2. Build the complete truth table and identify 1-rows.
3. Write canonical SOP, sketch the AND–OR circuit and list tests.

Requirement: authorisation AND exactly one key.

EXIT TICKET · 4 MIN

Week 5 checkpoint: five views must agree



Forward link: minterm indices become K-map cell addresses.

WEEK 6 · MEETING 1

Week 6 · Karnaugh maps and minimum SOP

Can four canonical terms become two without changing the function?

Four canonical terms can become two

WEEK 5

CANONICAL

A'BC' + A'BC + AB'C + ABC

4 terms · 12 literals

WEEK 6

MINIMUM SOP

A'B + AC

2 terms · 4 literals

K-map adjacency shows which variables can disappear.

WEEK 6 OUTCOMES

This week: map, group, minimise and verify

01 **Map**

Place truth-table or minterm data into 2–4 variable K-maps.

02 **Group**

Use powers of two, adjacency, wrap-around and overlap.

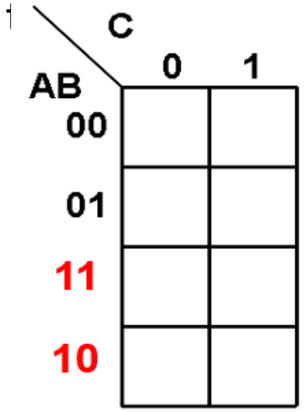
03 **Minimise**

Retain only literals that stay constant in each group.

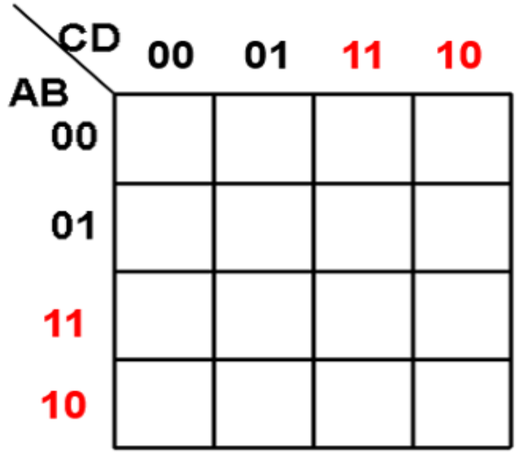
04 **Verify**

Compare implementation cost and prove equivalence.

- K-Map is similar to the truth table, but it **presents all the possible values of input and output**.
- This is shown in an **array of cells**.
- K-Maps can be used for expressions with **2, 3, 4 or 5** variables.
- The number of cells in a K-Map = total number of possible input variable combinations as in truth

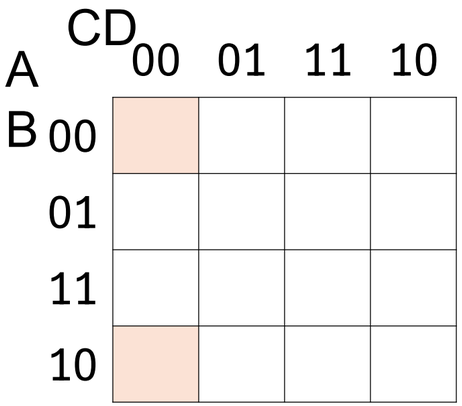


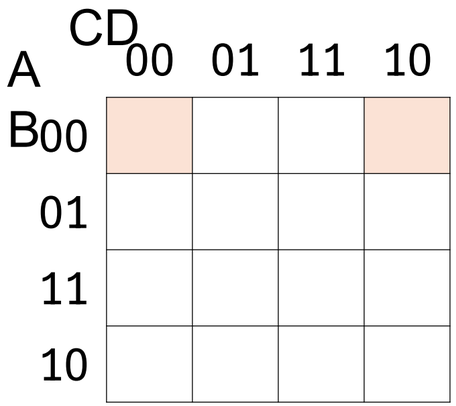
3 variables K-Map

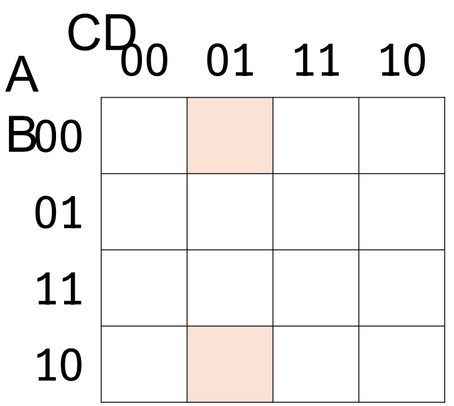


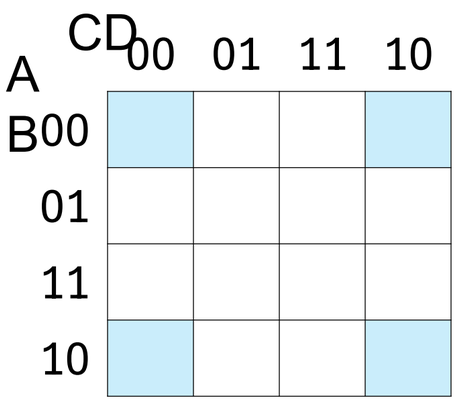
4 variables K-Map

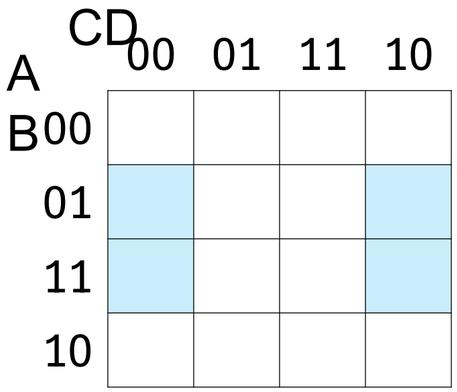
- In a K-map with 4-variable or more, the top-most & bottom-most cells of a column (and row) are **adjacent**.
- (*recall → cells that differ by only one variable are adjacent*)

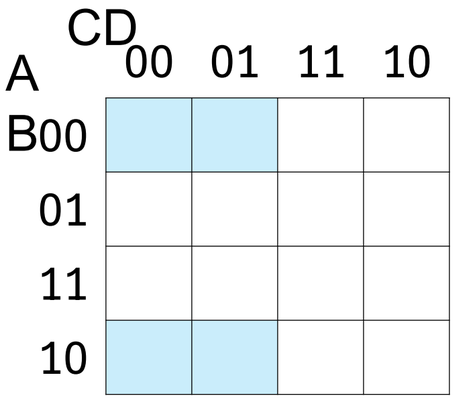


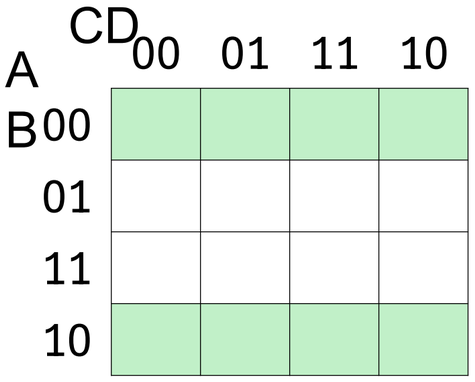


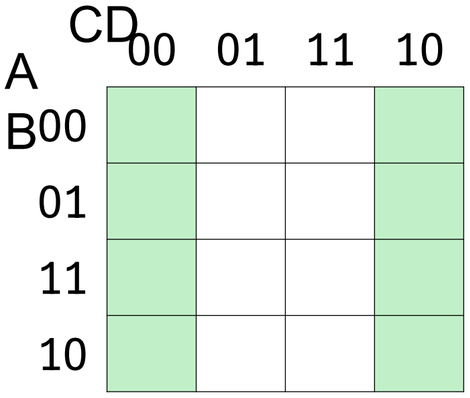












2 cells 4 cells 8 cells

Our anchor function returns as a K-map

ANCHOR FUNCTION

$$F(A,B,C) = \Sigma m(2,3,5,7)$$

Place 1 in cells m2, m3, m5 and m7. Every other cell receives 0.

Gray order keeps neighbours one bit apart

00
1 bit changes

01
1 bit changes

11
1 bit changes

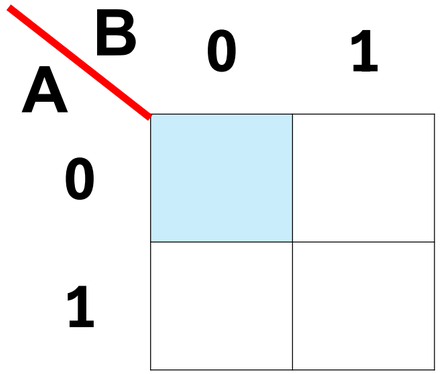
10
wraps to 00

K-map order is 00, 01, 11, 10 — not 00, 01, 10, 11.

First and last positions are adjacent.

A 2-variable map rearranges a truth table

A \ B	0	1
0	m0 - 00	m1 - 01
1	m2 - 10	m3 - 11



Same four rows · spatially arranged for adjacency

Adjacency means exactly one input bit changes

SHARED EDGE (BERKONGSI SISI)

VALID

01 ↔ 11

DIAGONAL (PEPENJURU)

INVALID

00 ↔ 11

OUTER COLUMNS (LAJUR PALING LUAR)

VALID WRAP

00 ↔ 10

Diagonal cells differ in two bits, so they are not adjacent. With columns ordered 00 01 11 10, the first and last columns wrap.

A group removes the variable that changes

2 CELLS

PAIR

m1 = 01
m3 = 11

A changes
B stays 1

→

RETAIN CONSTANT

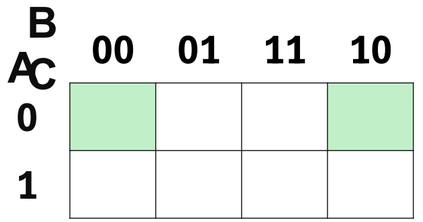
GROUP TERM

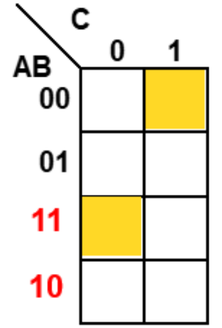
Term = B

Changing variables disappear; constant variables remain.

A 3-variable map uses Gray order across BC

A \ BC	00	01	11	10
0	m0	m1	m3	m2
1	m4	m5	m7	m6





Top row: m0, m1, m3, m2 · bottom row: m4, m5, m7, m6

Place the selector's 1s before grouping

A \ BC	00	01	11	10
0	m0 0	m1 0	m3 1	m2 1
1	m4 0	m5 1	m7 1	m6 0

FIRST

PLACEMENT

1 → m2, m3, m5, m7

0 → all other cells

Gray order changes position—not the minterm index.

Each pair produces one retained-literal term

A \ BC	00	01	11	10
0	m0 0	m1 0	m3 1	m2 1
1	m4 0	m5 1	m7 1	m6 0

BLUE

PAIR 1

(m2,m3) → A'B

GREEN

PAIR 2

(m5,m7) → AC

Minimum SOP: F = A'B + AC

BLOCK 1 CHECK · 8 MIN

Practice: minimise Σm(1,3,4,5)

1. Place 1 in m1, m3, m4 and m5 on a 3-variable map.
2. Form the largest valid groups using Gray adjacency.
3. Derive each term and write the minimum SOP.

Answer format: group → retained literals → final SOP.

WEEK 6 · MEETING 1

Block 2 · Four-variable maps

Gray order now appears on both axes.

A 4-variable map uses Gray order on both axes

AB \ CD	00	01	11	10
00	m0 m0	m1 m1	m3 m3	m2 m2
01	m4 m4	m5 m5	m7 m7	m6 m6
11	m12 m12	m13 m13	m15 m15	m14 m14
10	m8 m8	m9 m9	m11 m11	m10 m10

Rows AB: 00,01,11,10 · columns CD: 00,01,11,10

Edges and corners are adjacent through wrap-around

AB \ CD	00	01	11	10
00	m0 1	m1 0	m3 0	m2 1
01	m4 0	m5 0	m7 0	m6 0
11	m12 0	m13 0	m15 0	m14 0
10	m8 1	m9 0	m11 0	m10 1

4 CORNERS

ONE WRAP GROUP

m0 · m2 · m8 · m10

Across left/right
and top/bottom edges

Visual separation on paper does not mean non-adjacency.

Valid group sizes are powers of two

1	2	4	8	16
4 literals	3 literals	2 literals	1 literal	constant 1

Rectangle only · no 0s · choose the largest useful groups

Four corners produce B'D'

AB \ CD	00	01	11	10
00	m0 1	m1 0	m3 0	m2 1
01	m4 0	m5 0	m7 0	m6 0
11	m12 0	m13 0	m15 0	m14 0
10	m8 1	m9 0	m11 0	m10 1

GROUP TERM

RETAIN CONSTANTS

A changes → remove
B=0 → B'
C changes → remove
D=0 → D'

F = B'D'

One map can reveal two large groups

AB \ CD	00	01	11	10
00	m0 1	m1 0	m3 0	m2 1
01	m4 0	m5 1	m7 1	m6 0
11	m12 0	m13 1	m15 1	m14 0
10	m8 1	m9 0	m11 0	m10 1

BLUE

GROUP 1

corners → B'D'

GREEN

GROUP 2

m5,m7,m13,m15 → BD

Minimum SOP: F = B'D' + BD

Retain only variables that stay constant

BEHAVIOR IN GROUP	ACTION	RESULT
Variable changes	Remove it	No literal
Variable stays 1	Retain normal	X
Variable stays 0	Retain complemented	X’

Larger groups remove more changing variables.

BLOCK 2 CHECK · 8 MIN

Checkpoint: minimise an eight-cell wrap group

1. Map m0,m1,m2,m3,m8,m9,m10,m11.
2. Form one eight-cell group across the top and bottom boundary.
3. State which variable stays constant and write the term.

Visual separation does not imply non-adjacency.

UTM
UNIVERSITI TEKNOLOGI MALAYSIA
UNIVERSITI TEKNOLOGI MALAYSIA
ASCEND
2030

WEEK 6 · MEETING 2

Block 3 · Optimisation choices

A valid cover must be complete, legal and minimum—not merely colourful.

Overlap is allowed when it creates useful groups

ALLOWED

OVERLAP

A 1 may appear in multiple groups.

Use overlap when it forms larger, lower-literal groups.

REQUIRED

ESSENTIAL GROUP

Covers a required 1 that no other prime implicant covers.

Cover every required 1; never include a 0.

A don't-care may be used or ignored

USE X	IGNORE X	NEVER
when it enlarges a group	when it gives no benefit	treat X as a mandatory 1

Truth-table X = irrelevant; circuit-node X = unknown or illegal.

A don't-care can collapse two terms into one

A \ BC	00	01	11	10
0	m0 0	m1 1	m3 1	m2 0
1	m4 0	m5 X	m7 1	m6 0

1 TERM

WITH X

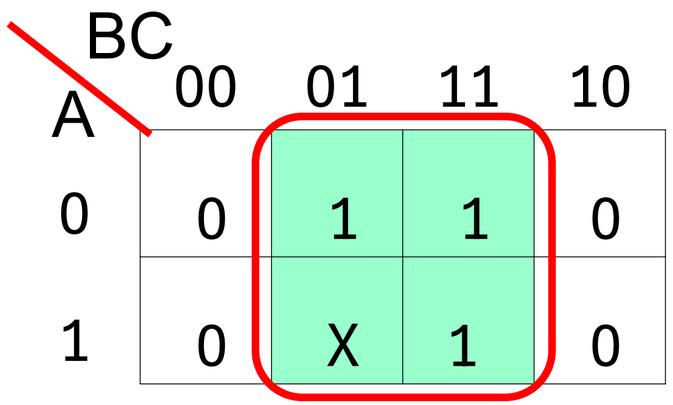
m1,m3,m5,m7 → C

2 TERMS

WITHOUT X

A'C + BC

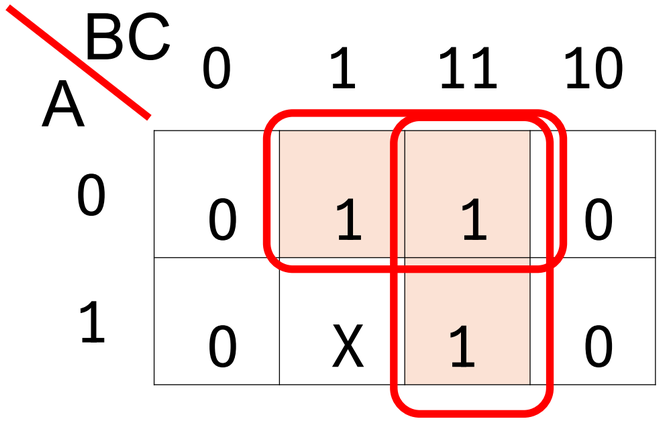
Use X only because it reduces the implementation.



1 TERM

WITH X

m1,m3,m5,m7 → **C**



2 TERMS

WITHOUT X

A'C + BC

Different covers can be equally minimal

VALID MINIMUM

COVER A

B'C + BC' + A'C'

VALID MINIMUM

COVER B

B'C + BC' + A'B'

F(A,B,C) = Σm(0,1,2,5,6). Minimum SOP may be non-unique; both covers use prime implicants.

Compare term count first, then literal count

COVER	PRODUCT TERMS	LITERALS	COST
A: B'C+BC'+A'C'	3	6	minimum
B: B'C+BC'+A'B'	3	6	minimum

Minimise lexicographically: fewest terms, then fewest literals.

PAIR ARGUMENT · 8 MIN

Choose and defend a minimum cover

1. Map each product term back to the cells it covers.
2. Confirm every required 1 is covered and no 0 is included.
3. Compare terms and literals; defend why both covers are minimum.

A valid answer includes evidence—not only the final expression.

BLOCK 3 CHECK

Audit every proposed K-map solution

01	**Coverage** Every required 1 is covered.
02	**Legality** No 0; rectangles; power-of-two sizes.
03	**Literals** Only constant variables are retained.
04	**Minimum cost** Compare complete covers by terms, then literals.

Largest group first is a heuristic—not proof of global minimum.

The minimum selector circuit is materially smaller

FORM	TERMS	LITERALS	AND PATHS	OR FAN-IN
Canonical SOP	4	12	4	4
Minimum SOP	2	4	2	2

Same truth table · fewer product paths · lower literal count

Minimum SOP follows the same verification discipline

01 MINIMUM SOP

02 AND–OR CIRCUIT

03 COST COUNT

04 SIMULATION

05 EQUIVALENCE

Optimise → implement → verify every row

Canonical and minimum outputs agree on all rows

A	B	C	Fcanonical	Fminimum	MATCH?
0	0	0	0	0	✓
0	0	1	0	0	✓
0	1	0	1	1	✓
0	1	1	1	1	✓
1	0	0	0	0	✓
1	0	1	1	1	✓
1	1	0	0	0	✓
1	1	1	1	1	✓

Eight matching rows prove equivalence for three inputs.

An XOR mismatch detector should remain 0

$$E = F_{canonical} \oplus F_{minimum}$$

PASS

E = 0

outputs match

FAIL

E = 1

outputs differ

Check all 2^N rows (‘perfect induction’): E must remain 0.

CAPSTONE · 14 MIN

Capstone: minimise and implement the alarm

1. Map Q=Σm(5,6,7) and form the largest valid groups.
2. Derive minimum SOP and sketch the two-path AND–OR circuit.
3. Compare cost, then verify canonical and minimum outputs for all rows.

Do not reveal your final form until every group is justified.

Module checkpoint: preserve behaviour, remove redundancy

01 MAP MINTERMS

02 DRAW GROUPS

03 DERIVE SOP

04 BUILD CIRCUIT

05 VERIFY ALL ROWS

$$F = \Sigma m(0,1,4,5,10,11,14,15)$$

Deliver: K-map · groups · minimum SOP · circuit · complete equivalence evidence

Mastery scope: 2–4 variable minimum SOP · POS grouping is not assessed.